SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

Domain 2

Prepared by Fantastic 5

Table of Contents

1. Introduction: ... 4
   1.1 Purpose: ... 4
   1.2 Background: ... 5
   1.3 Target Stakeholder ... 6
2. System Architecture & Design ... 7
   2.1 High Level Architecture ... 7
      2.1.1 Overview: ... 7
      2.1.2 LLM as Service Layer ... 7
      2.1.3 Dependency Diagram ... 7
      2.1.4 Sequence Diagram ... 8
   2.2 Technological Stack ... 9
      2.2.1 Frontend ... 9
      2.2.2 Backend ... 9
      2.2.3 Database ... 9
      2.2.4 Cloud / Deployment ... 9
   2.3 Key Data Flows ... 9
      2.3.1 Data Flow Diagram (DFD Explanation) ... 9
      2.3.2 Normalized Database Schema ... 10
3. Functional Requirements & Scope ... 11
   3.1 Minimum Viable Product (MVP): ... 11
   3.2 Non-Functional Requirements (NFRs) ... 11
   3.3 Out of Scope / Future Enhancements ... 12
4. Monitor, Evaluation, Assumptions & Dependencies ... 13
   4.1 Technical Evaluation: ... 13

4.1.1 Grayscale Rollout & A/B Testing: 13
4.1.2 Emergency Rollback (ER): 13
4.1.3 Priority Matrix: 13
4.2 Monitoring: 13
4.3 Assumptions: 13
4.4 External Dependencies: 14
5. Project Management & Team Contributions 15
5.1 Project Timeline 15
5.2 Team Members Role 15
5.3 Recommendations 15

1. Introduction:

FinSight AI is an AI-powered financial decision intelligence system designed to help users evaluate purchases, simulate financial outcomes and receive explainable recommendations. The system integrates structured financial data with Z.ai GLM to enable intelligent, context-aware decision-making.

1.1 Purpose:

The system analysis document highlights the technical architecture, data flow, AI model integration, and system design decisions behind the development of FinSight AI.

The key elements of the system covered are as follows:

1. **Architecture:**
   A client-server-based system with AI service integration which is Z.AI GLM and modular components deployed via cloud or local environment.
2. **Data Flows:**
   User inputs (financial data, purchase details, OCR data) are processed through backend logic, passed to AI services, and returned as structured insights.
3. **Model Process:**
   The system demonstrates end-to-end workflow of financial decision-making including input analysis, simulation, recommendation generation, and explanation.
4. **Role of Reference:**
   This document serves as a reference for developers and testers to understand the system architecture, workflows, and technical decisions.

1.2 Background:

Many individuals lack tools that provide intelligent financial guidance. Existing budgeting apps track expenses but fail to offer actionable recommendations or predictive insights.

1. **Previous Version:**

   Traditional financial tools:

   - Manual tracking
   - No AI integration
   - No decision-making support

2. **Changes in Major Architectural Components**

   - Introduction of AI reasoning layer which is Z.AI GLM
   - Integration of simulation engine
   - Addition of OCR for receipt analysis

3. **New Capabilities Introduced**

   - Real-time decision recommendation which will show BUY / DELAY / RECOSIDER
   - Financial scenario simulation
   - Explainable AI outputs
   - Peer benchmarking

1.3 Target Stakeholder

Stakeholders	Roles	Expectations
Users (Student / Professionals)	Input financial data evaluate purchases.	Accurate recommendations, simple UI
Development Team	Build and maintain system.	Modular architecture, scalable design.
QA Team	Test system reliability.	Complete test coverage, edge case handling.
Judges/ Stakeholders	Evaluate the system.	Clear logic, real-world applicability.

2. System Architecture & Design

2.1 High Level Architecture

2.1.1 Overview:

Type	Details
System	Web-based Application (Streamlit)
Architecture	Client-Server with AI Service Layer

FinSight AI is structured as a client-server system where:

- Frontend (Streamlit UI) handles user interaction.
- Backend (Python logix) will be processing the financial data.
- AI Service Layer (Z.AI GLM) will be performing the reasoning.
- Optional Storage which will be storing the datasets and inputs.

2.1.2 LLM as Service Layer

Z.AI GLM is integrated as a core service layer which is not just a module.

2.1.3 Dependency Diagram

a. Prompt Construction:

With the combination of financial data and decision output which will be formatted into a structured prompt.

b. Context Window:

Includes:

- User income
- Savings
- Purchase Price
- Simulations results

c. Response Handling:

GLM output → parsed → displayed in UI

d. Token Limitation Handling:

- Input truncated if too large
- Data summarized before sending

e. API Calss:

- Frontend → Backend
- Backend → GLM API
- Backend → Dataset

f. Component Interaction:

UI → Decision Engine → Simulation → GLM → Output

g. Distributed Components:

- Frontend
- Backend logic
- External AI API

2.1.4 Sequence Diagram

Purchase Decision Flow

1. User enters financial data and item price
2. Frontend sends data to backend
3. Backend computes decision logic
4. Simulation engines run projections
5. Data sent to GLM API
6. GLM return explanation
7. UI displays recommendations with explanation

2.2 Technological Stack

2.2.1 Frontend

Streamlit (Python-based UI)

2.2.2 Backend

Python (core logic, simulation engine)

2.2.3 Database

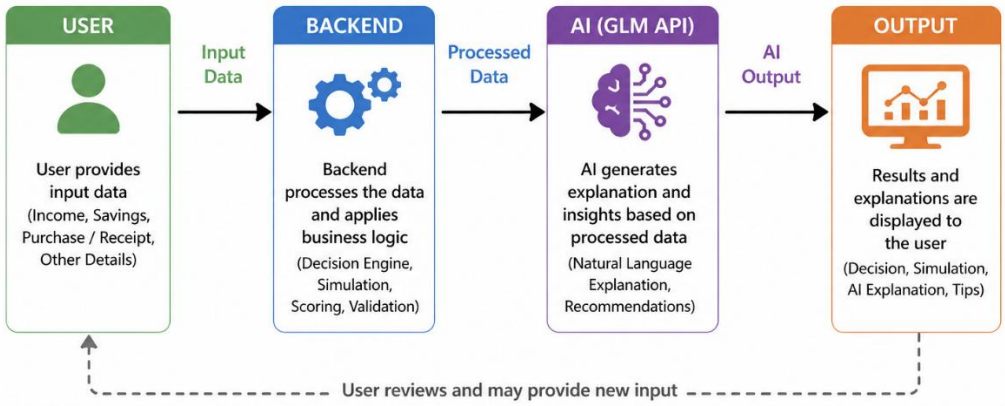
CSV datasets / local storage

2.2.4 Cloud / Deployment

Localhost / optional cloud hosting

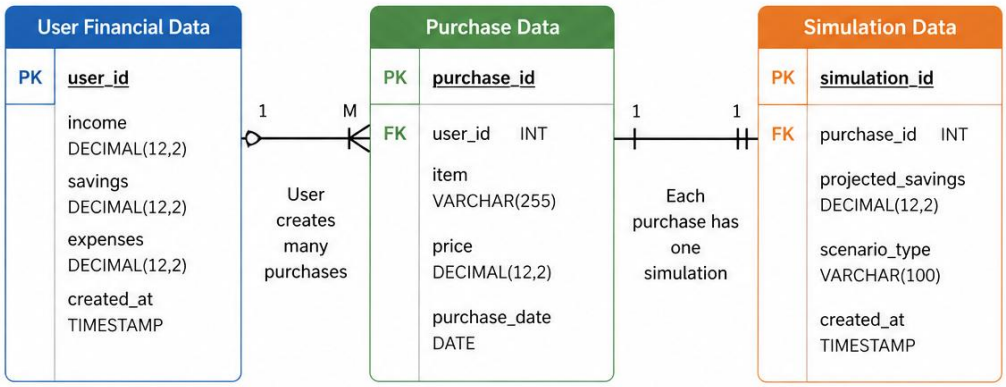
2.3 Key Data Flows

2.3.1 Data Flow Diagram (DFD Explanation)



- User → Input data
- Backend → Processes logic
- AI → Generates explanation
- Output → Displayed to user

2.3.2 Normalized Database Schema



Entities:

- User Financial Data

  Income, savings, expenses
- Purchase Data

  Item and Price
- Simulation Data

  Projected Savings

Relationships:

- User → Purchase (1:M)
- Purchase → Simulation (1:1)

3. Functional Requirements & Scope

3.1 Minimum Viable Product (MVP):

#	Features	Description
1	Purchase Decision Engine	Evaluate purchases and return BUY / DELAY/ RECONSIDER
2	Simulation Engine	Predict financial outcome over time
3	AI Explanation Engine	Generate explainable insights using GLM
4	OCR Receipt Scanner	Extract price from image input
5	Peer Benchmarking	Compare financial behavior

3.2 Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Scalability	Handle multiple users	Lightweight architecture
Reliability	Accurate recommendation output	Rule-based validation
Maintainability	Modular code	Separated components
Token Latency	Less than 3s response	Optimized prompts
Cost Efficiency	Low token usage	Input truncation and caching

3.3 Out of Scope / Future Enhancements

a. Bank API integration
b. Mobile application
c. Real-time financial tracking

4. Monitor, Evaluation, Assumptions & Dependencies

4.1 Technical Evaluation:

4.1.1 Grayscale Rollout & A/B Testing:

- Deploy features incrementally
- Compare recommendations strategies

4.1.2 Emergency Rollback (ER):

- Revert to previous stable version if there was being failure occurred

4.1.3 Priority Matrix:

Priority	Condition	Action
P1 Critical	AI output incorrect	Immediate fix
P2 Medium	Slow response	Optimization
P3 Low	UI issues	Minor fix

4.2 Monitoring:

Track:

- API response time
- AI output accuracy
- Error rates

4.3 Assumptions:

a. Users provide accurate financial data
b. Internet connection is stable
c. GLM API is available

4.4 External Dependencies:

Tools	Purpose	Risks
Z Ai GLM API	Core AI reasoning	API failure risk
OCR Library	Extract receipt data	Accuracy issues

5. Project Management & Team Contributions

5.1 Project Timeline

Day	Task
Day 1 - 2	Planning and idea validation
Day 3 – 5	Core feature development
Day 6 – 8	AI integration
Day 9 - 10	Testing and refinement

5.2 Team Members Role

Member	Role
Bibianne Zheyee Joseph	Data Engineer
Hang Xiu Jun	AI / GLM Engineer
Kishanea A/P Jeyakumar	Decision Engineer
Liew Jia Xin	Frontend (Streamlit UI)
Ng Guan Yik	Insight & Analytics Engineer

5.3 Recommendations

- Use caching for repeated AI queries.
- Optimize prompt size.
- Improve C=OCR accuracy with preprocessing.